CSC1101 Concurrency Assignment

Members:

Shane Mahon - shane.mahon25@mail.dcu.ie - 22376743
Sean Sweeney - sean.sweeney47@mail.dcu.ie - 22408204

Group: BT

VideoLink:

https://drive.google.com/file/d/1O8WPFipXiDSIeWkAdDUzlb_EiTnYnepe/view?usp=sharing

Table of Contents

1. Functionality.....2
2. Division of Work.....4
3. Running the code.....4
3. 1 Configuration:.....4
3. 2 Main parameters:.....4
4. Tasks / Dependencies.....5
4.1 Main Tasks.....5
4. 2 Shared dependencies.....5
4. 3 Dependency flow.....5
5. Patterns / Strategies.....5
5. Issue description.....6
5. 1 Fairness, Starvation Prevention, and Progress.....6
5.2 Progress Over Time.....6
5. 3 Deadlock / Livelock Control.....6

1. Functionality

DESCRIPTION	STATUS	NOTE
More than 1 stocker thread	Working	Multiple stockers work concurrently, no deadlock
Multiple Picker threads	Working	Multiple pickers run in parallel and complete attempts.
Delivery thread	Working	Deliveries generated by per tick probability model
Tick-based time model (1 day = 1000 ticks)	Working	No pick attempts happens after tick 1000
Seeded randomness	Working	Seeded RNG is used, concurrent scheduling can still change exact log ordering
Deliveries occur with per-tick probability 1/100	Working	Bernoulli delivery model with delivery prob = 0.01 per tick
Each delivery contains exactly 10 boxes	Working	Correct delivery payload
Random delivery box-type composition across section types	Working	Random assignment observed
Warehouse sections by type (configurable set of section types)	Working	Sections created from configurable type list. Verified
Initial stock of 5 boxes per section	Working	Inventory is not zero before first delivery, default set to 5
Staging area has unlimited capacity	Working	Staging never blocks because of size, no staging capacity limit enforced
Trolley system required for all work (pickers + stockers must acquire before acting)	Working	Work events occur only while thread holds trolley
Shared trolley pool with configurable K	Working	Verified across multiple k configurations
Trolley capacity limit of 10 boxes	Working	The total load never exceeds 10
Trolley release only when empty	Working	Remaining load always 0 on release
Deadlock prevention via reserving at least 1 trolley for stockers	Working	Reserve + borrow policy prevents picker-only trolley stalls while preserving stocker access.
Stockers cannot return boxes to staging after pickup	Working	Remaining load only decreases by stocking, no put-back path exists
Stocker movement timing: 10 ticks + 1 tick per box carried	Working	Tick gaps between moving match load dependent timing
Stocking timing: 1 tick per box placed	Working	Stock_end minus stock_begin tick equals number of boxes placed
Picking timing: 1 tick per box removed	Working	After wait ends, pick_done occurs 1 tick later
One stocker per section	Working	Section stocking exclusivity enforced
Pickers blocked while a section is being stocked	Working	Pick_done occur only after stock_end when overlap happens
Concurrent picking allowed when section is not being stocked	Working	Multiple pick_done events for same section close together

Pick attempt chooses random section once and cannot change during the attempt	Working	Section for pick_start.section matches pick_done.section for same pick_id
Pickers wait if chosen section is empty until stock becomes available	Working	Positive waited_ticks observed. waits resolve when section becomes available.
Limited section capacity enforced (configurable SECTION_CAPACITY)	Working	No section exceeded capacity and none went negative, enforced in section logic
Logging to terminal in machine parseable single line key=value format	Working	All logs are single line space separated key=value pairs
Logged events: delivery_arrived	Working	Event appears with correct per section counts
Logged events: acquire_trolley / release_trolley (with trolley_id, waited_ticks, remaining_load)	Working	All required fields present, remaining load always 0 on release
Logged events: stocker_load	Working	Per-section counts and total_load logged correctly
Logged events: move	Working	From, to, load, and trolley ID present
Logged events: stock_begin / stock_end (with stocked and remaining_load)	Working	Stocked and remaining load values present and consistent
Logged events: pick_start / pick_done (unique pick_id; waited_ticks on pick_done)	Working	Pick id is unique, pick_start_total is equal to pick_done_total across all seeds
Progress guarantee (no deadlock; system continues making progress)	Working	No stuck runs, pick_start_total is equal to pick_done_total
Clean shutdown at end-of-day: stop starting new pick attempts at tick >= SIM_TICKS and drain in-flight work before exit	Working	No late pick starts, in flight picks drain before exit
Configurable parameters via system properties (threads, K, reserve, capacities, probabilities, seed, run length)	Working	Different parameters successfully used in different test runs
Prioritisation of stocking boxes	Working	3 policies: Demand Aware, Round Robin and Lowest stock. Demand aware performed best so it's set as default.
Pick attempt rate (~100/day combined)	Partially working	Target is ~100/day but pick_starts_day=75.35 on average (20 runs). Trolley queues bring it down.
Partial stocking support	Working	Observed in logs
Clock API (nowTick(), sleepTicks(n) implemented	Working	Works cleanly, aligns with action timing checks
Staging exclusivity (one stocker at a time)	Working	Staging access serialised
Staging pickup cost is 1 per take action	Working	1 tick sleep after staging take
Stocker full-section behaviour releases exclusivity before waiting/redirecting	Working	stock_wait_full_begin/end events confirm behavior
Split trolley acquisition policy picker/stocker with reserve enforcement	Working	Passes
Stocker takes break of 150 ticks every 200-300 ticks	Working	Each stocker schedules random break interval, finishes its current trolley cycle first, then takes a 150-tick break. break_begin/break_end logged
Configurable stocker policy	Working	Stocker can use different section selection strategies, enabling comparative analysis of throughput, picker waiting time, and fairness

2. Division of Work

Nearly all of the project work was done together in person, so the design, implementation, testing, and refinement were highly collaborative.

Shane: worked on the core warehouse concurrency logic,
contributed to picker, stocker, trolley, and section coordination,
helped test correctness, validate log invariants, and organise evidence from runs,
helped prepare the report, functionality table, and video material.

Sean: worked on the stocker policy, trolley and clock design and implementation,
contributed to testing across multiple seeds and policy comparisons,
helped analyse results and compare stocker policy performance,
helped prepare the report, functionality table, and video material.

3. Running the code

The project is plain Java, no external libraries.

1. Open a terminal and go to the project root folder.
```
mkdir -p out
javac -d out src/warehouse/*.java
java -cp out warehouse.Main
```
2. To save outputs to a log file:
```
java -cp out warehouse.Main > run.log
```
3. Parameters can be overridden at runtime using JVM system properties
   E.g `java -cp out -DNUM_PICKERS=8 -DNUM_STOCKERS=3 warehouse.Main`

3. 1 Configuration:

The program supports two configuration sources:
1. warehouse.properties file (default in project root)
2. JVM system properties passed with -D...

Priority order is:
- -D system properties override file values
- file values override hardcoded defaults
- You can point to a custom config file by using:

`java -cp out -DCONFIG_FILE=/path/to/custom.properties warehouse.Main`

3. 2 Main parameters:

The main configurable keys are:
SIM_DAYS, **TICK_TIME_MS**, **SECTION_TYPES**, **INITIAL_STOCK_PER_SECTION, SECTION_CAPACITY**, **NUM_PICKERS**, **NUM_STOCKERS**, **K**(trolley count), **STOCKER_RESERVE**, **STOCKER_CYCLE_CUTOFF_TICKS**, **STOCKER_POLICY**, **STOCKER_BREAK_MIN_TICKS, STOCKER_BREAK_MAX_TICKS, STOCKER_BREAK_DURATION_TICKS, TROLLEY_CAPACITY**, **DELIVERY_PROB**, **BOXES_PER_DELIVERY**, **RANDOM_SEED**

Invalid configs fail fast: **TICK_TIME_MS** < 50, empty **SECTION_TYPES, K** < 2,
K <= STOCKER_RESERVE

4. Tasks / Dependencies

The program models the warehouse as concurrent tasks with shared resource dependencies.

4.1 Main Tasks

- DeliveryThread: Generates deliveries using a per-tick Bernouli trial (DELIVERY_PROB) and places boxes into staging.
- StockerThread (S1..Sn): Acquires trolley, takes boxes from staging, moves between locations, stocks sections, releases trolley only when empty, and takes breaks for 150 ticks. The program supports interchangeable stocker decision rules.
- PickerThread (P1..Pn): Waits between attempts, acquires a trolley, fixes a section for the attempt, waits if unavailable, picks exactly one box, releases the trolley.
- Main: Starts all threads, enforces day cutoff, closes new pick attempts, drains in progress picks, and joins threads.

4. 2 Shared dependencies

TrolleyPool: Both pickers and stockers depend on trolley availability before work can proceed.
StagingArea: Deliveries items into staging; stockers consume from staging; staging take is serialised. (one stocker at a time).
Section: per-section coordination for stocking, picking, empty/full waiting
Clock: All actions (move, take, stock, pick, waits, stocker breaks) depend on tick based timing.
Warehouse (lifecycle state): Threads depend on run state and attempt lifecycle gates (acceptNewAttempts, inFlightPicks) for safe shutdown.
EventsLogger: All tasks depend on synchronized logging to preserve machine parsable, one line event records.

4. 3 Dependency flow

- Delivery → Staging → Stocker → Section → Picker
- Trolleys are a cross cutting dependency for stockers and pickers.
- Section availability (empty, full, stocking) creates dynamic blocking dependencies between stockers and pickers.

5. Patterns / Strategies

- **Thread per role**
  Separate worker threads for delivery, stockers and pickers
- **Monitor pattern for shared state**
  Each shared component **(Section, TrolleyPool, StagingArea)** encapsulates its own lock and conditions.
- **Lock + condition coordination**
  Uses Reentrantock and Condition instead of spin waiting.
- **Producer consumer pipeline**
  Delivery produces to staging, stockers move stock to sections, pickers consume section stock.
- **Resource pool + reservation policy:**
  Trolleys are pooled, with reserved capacity for stockers (STOCKER_RESERVE) and controlled borrowing logic.
- **Atomic state management**

AtomicBoolean , AtomicInteger, and AtomicLong for shared flags and counters with low contention.

- **Cooperative cancellation**
  Threads observe shared running state and use interruptible waits/sleeps for graceful stop.
- **Two-phase shutdown/quiescence**
  Stops new pick attempts, waits for in-flight work to drain, then join() threads.
- **Serialized logging**
  Synchronised logger to prevent interleaved log lines from multiple threads.

5. Issue description

5.1 Fairness, Starvation Prevention, and Progress

The solution uses lock/condition coordination and bounded resources rules to keep actors moving and avoid permanent blocking.

Fairness:

- **TrolleyPool** serves both stockers and pickers through explicit acquire paths.
- A stocker reserve prevents pickers from acquiring all trolleys.
- A controlled borrow rule allows pickers to borrow into reserve when stockers are blocked on full sections
- Section access is coordinated so stockers get exclusive stocking windows, while pickers proceed concurrently when stocking is not taking place.

Starvation Prevention:

- **Pickers:**
  - Pickers wait on section conditions (empty or stockingInProgress) and are signaled on state changes.
  - Each attempt keeps its chosen section fixed.
  - In-flight attempt accounting ensures started attempts complete before shutdown
- **Stockers:**
  - Stockers can wait for section space without holding section exclusivity.
  - They can continue with remaining cargo and do not release tolleys early.
  - Staging pickup uses per-type caps (based on free space snapshot) to reduce unplaceable cargo building up near shutdown
  - Stockers take scheduled breaks only after finishing the current trolley cycle.

5.2 Progress Over Time

- Delivery continues stochastically, adding new work to staging.
- Stockers repeatedly move stock from staging into sections with movement + stocking costs.
- Pickers keep generating attempts with a target-rate schedule and complete picks as sections become available.
- Conditions are signaled on relevant state changes (Delivery arrivals, stock additions, picks) so blocked threads are reawakened.
- Shutdown is controlled; no new picks after day end, in-flight picks drain, supplier behavior and trolley policy avoid terminal waiting cycles.

5.3 Deadlock / Livelock Control

- Fixed resource rules (hold trolley until empty, no release with load)
- Section waiting path does not keep stocking exclusivity while full.
- Reverse+borrow trolley policy avoids full section standstills.